

WEIR - Assignment 2

Maj Bregar, Gal Fajon, Alen Leban
Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana

Abstract—This assignment builds on the PA1 crawl by extracting meaningful content from HTML pages, segmenting it into coherent text chunks, encoding the chunks with a sentence embedding model, and storing them in a pgvector-enabled PostgreSQL database for semantic retrieval.

We implement parsing tailored to 24ur.com articles, apply sentence-aware chunking, index embeddings with HNSW, and provide a query and reranking pipeline based on vector similarity and a cross-encoder reranker.

I. INTRODUCTION

This report describes the design and implementation of a parsing and semantic retrieval pipeline that processes the HTML pages collected by the preferential crawler in PA1. The goal is to extract clean, domain-relevant text from 24ur.com, segment it into meaningful chunks, and store vector embeddings in a pgvector database to support domain-specific question answering.

The system consists of four stages: filtering and parsing HTML pages using XPath and regular expressions, chunking the cleaned text into sentence-aware segments, embedding each segment and persisting vectors into a pgvector-backed table, and retrieving and reranking relevant segments for user queries.

II. METHODOLOGY

A. Website filtering and data source

We reuse the PA1 crawl database from the previous assignment and restrict processing to HTML pages with non-empty `html_content`. In our pipeline this filtering is performed in the database query that fetches page IDs and HTML bodies.

B. HTML parsing with XPath and regex

HTML parsing is implemented with `lxml`. We extract the article title by first reading the OpenGraph meta-data (`//meta[@property='og:title']/@content`); if missing, we fall back to `//title`. A regex post-process removes the site suffix “| 24ur.com” and normalizes whitespace. The main article body is extracted from the `#article-body` node using XPath, and non-content elements (`script`, `style`, `noscript`, `img`, `figure`, `svg`, `picture`) are removed to keep only the textual content.

We additionally capture the lead summary by locating the main article picture and the adjacent summary paragraph via XPath, then prepend it to the article body. Regex is used for lightweight text normalization (whitespace collapsing) and title cleanup.

This hybrid approach keeps extraction robust while remaining lightweight and transparent.

C. Text chunking strategy

We split the extracted text using NLTK’s Slovene sentence tokenizer and build chunks by accumulating complete sentences until a configurable maximum word count is reached (`CHUNK_LENGTH` in the environment, default value 256). This preserves sentence boundaries and keeps each segment topically coherent.

D. Database schema and pgvector indexing

We extend the PA1 database with a `cleaned_content` column and three segment tables for different embedding sizes: `page_segment_vec384`, `page_segment_vec768`, and `page_segment_vec1024`. Each segment row stores the page ID, the model ID, the text segment, and the vector embedding. A separate `model` table records the embedding model name, enabling multiple models to coexist.

For fast similarity search we add HNSW indexes for cosine and L2 distances, plus a `model_id` index to speed up model-specific filtering.

E. Embedding models

Embeddings are generated using transformer models. Our initial configuration used `sentence-transformers/all-MiniLM-L6-v2[?]` and mean pooling over the token embeddings with L2 normalization. When evaluating performance, we used multiple models, including `sentence-transformers/all-mpnet-base-v2[?]`, `BAAI/bge-m3[?]`, `EMBEDDIA/slobera[?]`, and `cjvt/crosloengual-bert-si-nli[?]`, to analyze quality and language coverage.

F. Querying and similarity metrics

Query vectors are computed with the same embedding model as the indexed segments and searched in pgvector using one of three metrics: cosine distance, L2 distance, or L1 distance. Our default configuration uses cosine distance.

G. Reranking

To improve relevance ordering, we apply a cross-encoder reranker on the top-*k* results returned by pgvector. The reranker scores each (query, segment) pair and reorders the results by predicted relevance. We use `cross-encoder/ms-marco-MiniLM-L6-v2` as the default reranker model.

III. EXPERIMENTS

IV. RESULTS

We compared multiple embedding models and dimensions using a small evaluation script that measures precision-at- k against keyword-based relevance heuristics and visualizes embedding clusters. The tested models include multilingual and Slovene-specific models.

We observed that multilingual models offer broader coverage, while Slovene-specific models can improve precision on domain-specific queries.

A. Retrieval Demo

The demo program takes a user query, computes its embedding, performs a similarity search over pgvector, and returns the top- k segments. We include three queries that retrieve relevant results for our domain and three queries that initially fail to retrieve the expected information. The failed queries are used in the reranking evaluation.

B. Reranking Evaluation

We apply a cross-encoder reranker to reorder the top- k segments. In our tests this improves the ranking of relevant segments when the initial embedding retrieval is noisy. We report three examples where reranking improves the result ordering and one example where reranking still fails to surface the expected information.

C. Limitations

The parser is tailored to 24ur.com and relies on stable HTML structure. Pages that deviate from the `#article-body` template or heavily script-rendered content can yield empty or partial text. Chunking relies on sentence tokenization quality and may produce uneven chunk sizes. Reranking improves accuracy but adds latency, which may be problematic for interactive settings.

D. Discussion

The pipeline satisfies the assignment requirements by combining parsing, sentence-aware chunking, pgvector indexing, and reranking. Its main strength is a transparent extraction procedure tailored to 24ur.com that preserves clean article text and yields coherent segments for retrieval.

There are several limitations. The parser targets the `#article-body` structure and will fail on pages that deviate from this template, yielding empty content. Chunking relies on sentence tokenization quality, which can be noisy in HTML-derived text.

Finally, cross-encoder reranking improves precision but increases query latency.

E. Examples and failure cases.

V. CONCLUSION

We implemented an end-to-end extraction and retrieval pipeline for the PA1 crawl of 24ur.com. The system parses raw HTML, chunks text using sentence boundaries, stores embeddings in a pgvector database with HNSW indexing, and

retrieves and reranks results for user queries. The approach is modular, supports multiple embedding models, and can be scaled to the full crawl by adjusting the parsing limit.

REFERENCES